

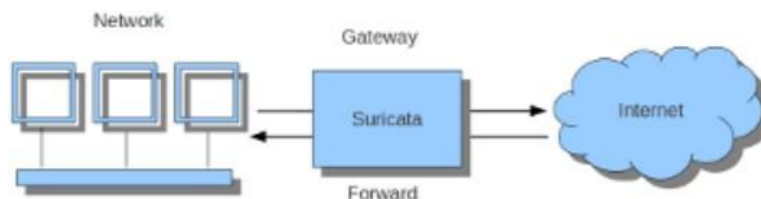
Network Intrusion Detection & Prevention Systems

Exploitation : Use of Suricata IPS

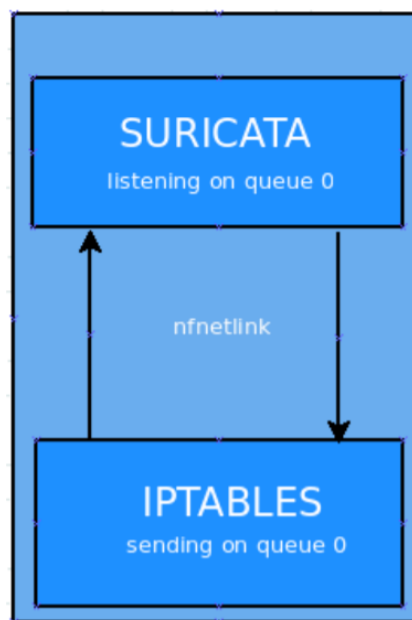
(Part 2)

I. Suricata IPS configuration

In this part of the Lab, we are going to configure Suricata as an inline IPS. For that, we need to configure the installed firewall (iptables) to forward all the traffic to Suricata to be treated before reaching the supervised network. Using this IPS, we will block any malicious traffic [1].



NFQUEUE is by suricata when working as an IPS. With NFQUEUE it is possible to delegate the verdict on the packet to a userspace software. The Linux kernel will ask a userspace software connected to a queue for a decision.



With NFQUEUE incoming packet matched by an iptables rule is sent to Suricata through nfnetlink. Suricata

receives the packet and issues a verdict depending

on our ruleset. The packet is either transmitted or rejected by kernel. Consequently, the action NFQUEUE will be used, instead of ACCEPT for the iptables rules. Using NFQUEUE, iptables will send the packets for separate and specific queue that will be used as an input for Suricata.

1. The interaction between iptables and Suricata will be configured in the section “nfq” of the file.

/etc/suricata/suricata.yaml as follows:

```
nfq:
mode: accept      #By default the packet will be accepted or dropped by Suricata
#repeat_mark: 1   #If the mode is set to 'repeat', the packets will be marked after being processed by Suricata.
#repeat_mask: 1
#route_queue: 2   #Here you can assign the queue-number of the tool that Suricata has to send the packets #to after processing
them.
```

- If the mode is set to 'accept', the packet that has been send to Suricata by a rule using NFQ, will not be inspected by the rest of the iptables rules after being processed by Suricata.
- If the mode is set to 'repeat', the packets will be marked by Suricata and then re-injected at the first rule of iptables. To mitigate the packets from being going round in circles, the rule using NFQ will be skipped because of the mark.
- If the mode is set to 'route', the packet will be sent to another tool after being processed by Suricata. It is possible to assign this tool at the mandatory option 'route_queue' and link every engine/tool to a different queue-number.

2. Check if NFQ is enabled in Suricata. For this purpose, use the following command:

```
#suricata --build-info
```

```
Suricata Configuration:
AF_PACKET support:      yes
eBPF support:           no
XDP support:            no
PF RING support:        no
NFQueue support:        yes
NFLOG support:          no
IPFW support:           no
Netmap support:         no
DAG enabled:            no
Napatech enabled:       no
WinDivert enabled:      no
```

3. Check the default rules and the default policy for the chains INPUT and OUTPUT

```
# iptables -L
```

4. To send all the incoming and outgoing traffic from iptables to Suricata, use the following commands:

```
# iptables -F  
# iptables -A OUTPUT -j NFQUEUE  
# iptables -A INPUT -j NFQUEUE
```

5. Open the Suricata configuration file `/etc/suricata/suricata.yaml` in your editor and scroll down to the "stream" section. There, set "inline" to "yes". This will force Suricata to do stream reassembly in an IPS aware way.

```
#  
stream:  
  memcap: 64mb  
  checksum-validation: yes      # reject incorrect csums  
  inline: auto                  # auto will use inline mode in IPS mode, yes or no set it statically  
  reassembly:  
    memcap: 256mb  
    depth: 1mb                  # reassemble 1mb into a stream  
    toserver-chunk-size: 2560  
    toclient-chunk-size: 2560  
    randomize-chunk-size: yes  
    #randomize-chunk-range: 10
```

II. Preventing against ICMP echo requests

6. Filtering ICMP echo requests can help conserve network resources by reducing unnecessary traffic. This is especially relevant in environments where bandwidth is limited. Execute In `/var/lib/suricata/rules/custom.rules` change the icmp detection rule as follows to block any icmp ping request datagram sent to server 8.8.8.8

```
# vi /var/lib/suricata/rules/custom.rules
```

```
drop icmp $HOME_NET any -> 8.8.8.8 any (icode:0; itype:8; msg: "Ping Detected & Blocked"; classtype:icmp-event;  
sid:1000001; rev:1;)
```

7. To run suricata as an IPS with the NFQ mode, you must make use of the `-q` option. This option tells Suricata which of the queue numbers it should use.

```
# suricata -c /etc/suricata/suricata.yaml -q 0
```

8. the following commands to test Suricata response.

```
# ping 8.8.8.8
```

```
# nslookup http://portquiz.net/
```

```
# curl portquiz.net
```

9. There should be no response to ping command, while DNS and HTTP connection should be possible. Check the log file `fast.log` and verify that suricata has successfully blocked the icmp traffic, logged the event and generated the alert.

```
# tail -f /var/log/suricata/fast.log
```

```
11/11/2021-10:03:30.922123 [Drop] [**] [1:1000001:1] Ping Detected [**] [Classification: Generic ICMP event] [Priority: 3] {ICMP} 10.0.2.15:8 -> 8.8.8.8:0
```

III. Filtering clear HTTP traffic

Many data protection regulations and standards, such as GDPR, require organizations to implement measures to protect the confidentiality of sensitive data. Filtering clear HTTP traffic is one way to enhance data protection and align with regulatory requirements.

10. In `/var/lib/suricata/rules/custom.rules` add the following rule to block any access to HTTP websites

```
# vi /var/lib/suricata/rules/custom.rules
```

```
drop tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"Dropped HTTP traffic"; priority:1; sid:1000002)
```

11. Stop suricata and start it again as an IPS with the NFQ mode

```
# suricata -c /etc/suricata/suricata.yaml -q 0
```

12. Using the command curl try to access to <http://portquiz.net/> through two different TCP ports 80 and 445 (usually used by SMB protocol). note that the HTTP server portquiz.net is especially listening to all TCP ports, allowing HTTP access through anyone of them. Argument the results that you obtain

```
[root@centos8 centos]# curl http://www.portquiz.net
curl: (6) Could not resolve host: www.portquiz.net
[root@centos8 centos]# curl http://www.portquiz.net:445
Port 445 test successful!
Your IP: 196.203.181.122
```

13. To prevent access to portquiz website through any possible port, we propose to update the previous rule as follows:

```
drop tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"Dropped HTTP traffic"; content:"User-Agent:"; http_header;
nocase; priority:1; sid:1000002; rev:1;)
```

14. Access again to portquiz website through the two TCP ports 80 and 445 using the browser Firefox. Now, the website is completely unreachable. Explain why?

<http://portquiz.net>

<http://portquiz.net:445>

15. Check the log file fast.log, and verify that suricata has successfully blocked the two HTTP connections (port 80 and port 445)

```
12/05/2023-15:04:11.743498 [Drop] [**] [1:1000002:1] Dropped HTTP traffic [**] [Classification:
(null)] [Priority: 1] {TCP} 10.0.2.15:57416 -> 35.180.139.74:445
12/05/2023-15:04:13.641910 [Drop] [**] [1:1000002:1] Dropped HTTP traffic [**] [Classification:
(null)] [Priority: 1] {TCP} 10.0.2.15:34762 -> 34.107.221.82:80
```

16. Using your browser Firefox try to access to the website <https://www.google.com>

Is access possible? Explain why?

IV. TLS analysis

17. Connect to the website <https://badssl.com/> and click on “expired” to access to the subdomain <https://expired.badssl.com/> that uses an expired certificate. You should click on accept the risk and continue if your browser asks for confirmation. Check that the validity period of the certificate has ended.
18. In the rule file custom.rules add the following entry to drop access to sites with expired certificates

```
drop tls any any -> any any (msg:"expired certs"; tls_cert_expired; priority:1; sid:1000005; rev:1;)
```

19. Stop suricata and start it again as an IPS with

the NFQ mode

```
# suricata -c /etc/suricata/suricata.yaml -q 0
```

20. Test that access to <https://expired.badssl.com/> is dropped and that the alert is generated.

21. In the rule file custom.rules add a new entry to drop access to facebook website using TLS, matching the content of facebook X509 certificate.

22. Now open Firefox, and try to go to <http://www.facebook.com/>. The request should time out.

23. Check the logfile `/var/log/suricata/fast.log` and verify the displayed alert message

V. Downloaded files collection

24. Open `suricata.yaml` configuration file, go to the output section, and

update the `file-store` variable to enable file extraction capability.

```
- file-store:
  version: 2
  enabled: yes

# Set the directory for the filestore. Relative pathnames
# are contained within the "default-log-dir".
dir: filestore
```

25. In `/var/lib/suricata/rules/custom.rules` add the following rule to store all files to disk.

```
alert http any any -> any any (msg:"FILE store all"; filestore; sid:1; rev:1;)
```

26. Stop and reexecute suricata using the command:

```
# suricata -c /etc/suricata/suricata.yaml -q 0
```

27. Download a GIF image through a plaintext HTTP connection

28. Go to the folder `Review` the folders `/var/log/suricata/filestore/` and execute the following commands. You should notice that one of the stored image is a GIF file.

```
# cd /var/log/suricata/filestore/
# file */*
```

```
[root@centos8 filestore]# file */*
4b/4bfe3adb36911561eefe2421a674465ab14a9f42879a986f71bc844612fa9981: GIF image data, version 89a, 249 x 72
```

29. Open the stored file using the command display (you should install

ImageMagick using yum if it is not already available in your system) and check the obtained image.

```
# display 4b/4bfe3adb36911561eefe2421a674465ab14a9f42879a986f71bc844612fa9981
```

VI. Generating and analyzing alerts in

JSON format

30. Suricata can output alerts, http events, dns events, tls events and file info through EVE json format. The JSON format allows a nice handling of data in external tool like Elasticsearch or Splunk. Open the Suricata configuration file `/etc/suricata/suricata.yaml` in your editor and scroll down to the "output" section and check that logging in eve-log format is enabled:

```
# Configure the type of alert (and other) logging you would like.
outputs:
  # a line based alerts log similar to Snort's fast.log
  - fast:
    enabled: yes
    filename: fast.log
    append: yes
    #filetype: regular # 'regular', 'unix_stream' or 'unix_dgram'

  # Extensible Event Format (nicknamed EVE) event log in JSON format
  - eve-log:
    enabled: yes
    filetype: regular #regular|syslog|unix_dgram|unix_stream|redis
    filename: eve.json
    # Enable for multi-threaded eve.json output; output files are amended with
```

31. Download the PCAP traffic capture of the WANNACRY ransomware attack which spreads using ETERNALBLUE exploit, and then unzip the pcap file (the password is: infected) using the two commands:

```
# wget https://raw.githubusercontent.com/tatsuiman/malware-traffic-
analysis.net/a5a261cacaf84079e4c0e0f2729c81226a27eca7/2017-05-18-
WannaCry-ransomware-using-EnternalBlue-exploit.pcap
```

32. Make sure that the rule file `suricata.rules` is activated in the configuration file `/etc/suricata/suricata.yaml`

```
default-rule-path: /var/lib/suricata/rules

rule-files:
  - suricata.rules
  - custom.rules
```

33. Read the pcap file using suricata, while disabling all checksum checking in the loaded datagrams

```
# suricata -r 2017-05-18-WannaCry-ransomware-using-EnternalBlue-
exploit.pcap -c /etc/suricata/suricata.yaml -l /var/log/suricata/ -k
none
```

Open the content of `fast.log` and read it. You can also search for keyword exploit in `fast.log` using the


```
# grep -in exploit /var/log/suricata/fast.log
```

34. While the json format is readable by human but as an event/record can contain a lot of data it can be difficult to do a by-eye analysis when looking at a file. Therefore, we will use the jq utility which is a tool dedicated to the transformation/parsing of a JSON entry. You can review the json formatted version of the log available in: <https://suricata.readthedocs.io/en/suricata-6.0.2/output/eve/eve-json-format.html>

35. Review all alerts using the command

```
# cd /var/log/suricata/
# jq . eve.json | less
```

36. To find alerts with a severity of 1 and display the source/destination IP and the alert, execute the following command:

```
# jq -c 'select(.alert.severity==1) | [.src_ip,.dest_ip,.alert.signature]' /var/log/suricata/eve.json | less
```

You can also group each alert, and display the highest count at the top of the list, using the command:

```
# jq -c 'select(.alert.severity==1) | [.src_ip,.dest_ip,.alert.signature]' /var/log/suricata/eve.json | sort | uniq -c | sort -rn | more
```

To view all the alerts and show the IP address and port information, execute the following command:

```
# grep '"event_type":"alert"' /var/log/suricata/eve.json | jq
'"\(.timestamp) | \(.alert.gid):\(.alert.signature_id):\(.alert.rev) |
\(.alert.signature) | \(.alert.category) | \(.src_ip):\(.src_port) ->
\(.dest_ip):\(.dest_port)"'
```

37. Among the displayed alerts is the following:

```
"2017-05-18T13:42:07.220702+0530 | 1:2025992:2 | ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic Flags) | A
Network Trojan was detected | 192.168.116.149:49368 -> 192.168.116.138:445"
```

Use the following command to determine the detection rule responsible for the alert generation

```
# grep -i "ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic
Flags)" /var/lib/suricata/rules/suricata.rules
```

```
[root@centos8 centos]# grep -i "ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic Flags)" /var/lib/suricata/rules/suricata.rules
alert smb any any -> $HOME_NET any (msg:"ET EXPLOIT Possible ETERNALBLUE Probe MS17-010 (Generic Flags)"; flow:to server,established; content:"|ff|SMB|25 00 00 00 00|"; offset:4; depth:9; content:"|00 00 00 00 00 00 00 00 00|"; distance:5; within:10; content:"|23 00 00 00 07 00 5c 50 49 50 45 5c 00|"; fast_pattern; endswith; threshold: type limit, track by_src, count 1, seconds 30; reference:url,github.com/rapid7/metasploit-framework/blob/master/modules/auxiliary/scanner/smb/smb_ms17_010.rb; classtype:trojan-activity; sid:2025992; rev:2; metadata:affected_product Windows XP_Vista_7_8_10_Server_32_64 Bit, attack_target Client_Endpoint, created_at 2018_08_15, deployment Perimeter, former_category EXPLOIT, malware_family ETERNALBLUE, signature_severity Major, updated_at 2019_09_28;)
```

Appendix

Suricata Rules Format

The Suricata rules/signature consists of the following parts:

- action: determines what happens when the signature matches (e.g., alert, drop)
- header : protocol + source address + source port -> destination address + destination port (defines the protocol, IP addresses, ports and direction of the rule)
- rule options: meta-settings + payload keywords + protocol keywords (defines the specifics of the rule)

Action: It is the first word in the signature, it can be drop/alert/pass... (for more details, visit <https://suricata.readthedocs.io/en/latest/configuration/suricata-yaml.html#suricata-yaml-action-order>)

Protocol: This keyword tells Suricata which protocol it concerns. It can be one of the four basic protocols: tcp, udp, icmp and ip (ip stands for 'all' or 'any') or one of so-called application layer protocols, as: http, ftp, tls (this includes ssl), smb, dns, ssh, smtp and more.

Source and destination: It specifies the traffic source and destination. Normally, you would also make use of variables, such as \$HOME_NET and \$EXTERNAL_NET. The configuration file specifies the IP addresses these concerns, and these settings will be used in place of the variables in you rules (see <https://suricata.readthedocs.io/en/latest/configuration/suricata-yaml.html#suricata-yaml-rule-vars> for more information).

Ports (source and destination): The first emphasized part is the source, the second is the destination.

Rule options: The rest of the rule consists of options. These are enclosed by parenthesis and separated by semicolons. Some options have settings (such as msg), which are specified by the keyword of the option, followed by a colon, followed by the settings. Others have no settings, and are simply the keyword (such as nocase):

<keyword>: <settings>;

<keyword>;

Flow: The flow keyword can be used to match on direction of the flow, to/from client or to/from server. It can also match if the flow is established or not.

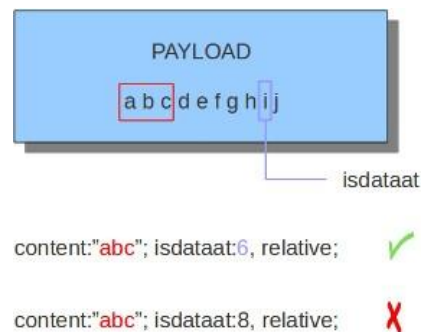
Content: The content keyword is very important in signatures. Between the quotation marks you can write on what you would like the signature to match. It is possible to use several contents in a signature.

Contents match on bytes. You can match on all

characters; But not all of the bytes are printable characters. For these bytes hexadecimal notations are used. Many programming languages use 0x00 as a notation, however the rule language uses |00| as a notation (|61| is a - |61 61| is aa - |41| is A - |21| is ! - |3A| is : and so on)

nocase: If you do not want to make a distinction between uppercase and lowercase characters, you can use nocase. The keyword nocase is a content modifier.

Isdataat: The purpose of the isdataat keyword is to look if there is still data at a specific part of the payload. The keyword starts with a number (the position) and then optional followed by 'relative' separated by a comma and the option rawbytes. You use the word 'relative' to know if there is still data at a specific part of the payload relative to the last match.



http_header: With the http_header content modifier, it is possible to match specifically and only on the HTTP header buffer. This contains all of the extracted headers in a single buffer.

Reference: The reference keywords direct to places where information about the signature and about the problem the signature tries to address, can be found. It has the following format: "reference: type, reference"

Classtype: The classtype keyword gives information about the classification of rules and alerts. It consists of a short name, a long name and a priority. For each classtype, the classification.config has a priority which will be used in the rule. (classtype definition: config classification: classtype, Alert, Priority)

sid (signature ID): The keyword sid gives every signature its own id. This id is stated with a number. The format of sid is: "sid:123;"

rev (revision): Represents the version of the signature. If a signature is modified, the number of rev will be incremented by the signature writers. The format of rev is: "rev:123;"

References

- [1] Pr. Slim Rekhis, Intrusion Detection: Network Intrusion Prevention Systems Exploitation Use of Suricata IPS (Part 2)

